07 — Reading a Research Paper, and Reproducing It

How to read a paper well enough to discuss it critically, demonstrated on FlashAttention, whose central claim you can check on a laptop.

Reading time: about 40 minutes. You need: modules 00 and 01, and section 2 of module 05. You do not need: a GPU.

Two answers to the same question

The interviewer sent you a paper three days ago. Now they ask: "What did you think of it?"

The first candidate walks through it section by section. The introduction motivates the problem, section 3 describes the method, the experiments show improvements. Everything they say is correct.

The second candidate says something like this: "The central idea is the online softmax, and the exactness results support it well. I am less convinced by the speed comparison, because the baseline was an unoptimised implementation of the time."

Both candidates read the paper. Only the second one thought about it. The interviewer already knows what the paper says, so the first answer tells them almost nothing. The second shows judgement, and judgement is what this interview round measures.

This article gives you a method for producing the second kind of answer. It then applies the method to FlashAttention, a paper about the attention function you have been building since module 00.

What you will learn

By the end of this article, you will be able to:

- read a paper in three passes, and know which pass interviews actually test;
- prepare answers to the questions interviewers ask about any paper;
- criticise a paper in proportion: specific, sourced and fair;
- explain FlashAttention from its key idea up, with numbers;
- present your own research in about 90 seconds.

Sections 1 to 4 cover method. Section 5 is the worked example. Section 6 covers your own research. A recap and the exercise briefing follow at the end.

---

1. What this interview round measures

In this round, you receive a paper a few days in advance and then discuss it for about 45 minutes. The paper might be a classic, a recent result, or one of the interviewer's own.

The round is hard to fake. It checks five things:

- **Comprehension**: can you state the contribution in two sentences, in your own words?
- **Judgement**: can you tell the central idea apart from the supporting detail?
- **Scepticism**: what would you need to see before believing the main claim?
- **Generativity**: what is the obvious next paper, and what breaks if an assumption changes?
- **Honesty**: can you say "I don't know", and then reason towards an answer out loud?

The most common failure is summarising. A summary proves you read the paper, not that you thought about it.

---

2. Reading in three passes

The best-known method comes from a two-page note by Srinivasan Keshav. It reads a paper in three passes, each deeper than the last.

Pass 1 takes five to ten minutes. Read the title, abstract, introduction, section headings, conclusions and the list of references. Afterwards, you should be able to say what problem the paper solves, and roughly how. Keshav suggests checking five things, which he calls the five Cs: the *category* of paper, its *context*, whether its assumptions look *correct*, its *contributions*, and its *clarity*.

Pass 2 takes about an hour. Read properly, but skip the proofs. Look hard at the figures, especially their axes, error bars and baselines. Note the references you should read. Afterwards, you should be able to explain the paper, with its evidence, to someone else.

Pass 3 takes several hours. Rebuild the work, either in your head or in code. Make the same assumptions and try to reach the same results. This is where you find the steps the authors left out, and the assumptions they did not know they were making.

Pass 3 is the one interviews are really about, and the one almost nobody does. This module's exercise is a pass 3 on FlashAttention.

---

3. Questions to answer before the interview

Write down an answer to each of these before you walk in. Most interview questions about a paper are versions of them.

1. What is the contribution, in two sentences, in your own words?
2. What is the key idea: the single insight that makes everything else follow?
3. What did the authors compare against, and was the comparison fair? Did they match compute, match parameter counts, or neither?
4. Which experiment carries the real weight of the argument? Which figure could be deleted?
5. What is the most important ablation, and is it there?
6. What would break the result: a larger scale, different data, or a changed assumption?
7. What does the paper *not* claim, that a quick reader might think it claims?
8. What is the obvious follow-up, and has someone already written it?
9. Which two or three papers is it really responding to?
10. With one week and eight GPUs, how would you reproduce the main result, and what would go wrong first?

---

4. Criticising in proportion

A good critique is specific, sourced and proportionate. Compare three ways of saying something about the same paper:

Register	Example	Problem
dismissive	"It's incremental."	cannot be checked or answered
credulous	"The results are impressive."	says nothing
calibrated	"The main idea is well supported by table 2. I'm less convinced by table 3, because it matches parameters rather than compute, which favours their method."	none

Save real scepticism for the places that deserve it. These are the usual ones:

- baselines that were not tuned as carefully as the new method;
- test sets that may have appeared in the training data;
- results shown at one scale but claimed for all scales;
- differences smaller than the variation between random seeds, from a single run;
- phrases like "we observe that" doing the job of "we tested whether".

---

5. Worked example: FlashAttention

The paper for this module is Dao et al. (2022), *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. It suits this module for three reasons. It is about the attention function from earlier modules. It is short and clearly written. And its main claim can be checked on a laptop, because it is a claim about exact arithmetic, not about a training run.

The contribution, in two sentences

Standard attention is slow mostly because it reads and writes a large score matrix, of size $T \times T$, to and from the GPU's main memory. FlashAttention computes exactly the same result without ever storing that matrix, which makes it both faster and far lighter on memory.

Notice the word "exact" in the title. Earlier work had proposed many *approximate* versions of attention, which traded accuracy for speed and often failed to run faster in practice. FlashAttention makes a stronger and easier-to-check claim: the same answer, with less data moved.

Why the matrix is the problem

Recall module 05. A GPU computes far faster than it can move data in and out of its main memory. It also has a small amount of very fast on-chip memory, called SRAM.

Standard attention computes the scores and writes the whole $T \times T$ matrix to main memory. It reads the matrix back to apply the softmax, and writes the result. Then it reads that result again, to multiply by the values. Each of those trips moves T^2 numbers. So the operation is limited by memory traffic, not by arithmetic.

The key idea: a softmax you can compute in pieces

To avoid storing the matrix, FlashAttention processes attention in small blocks that fit in SRAM. The difficulty is the softmax. Its denominator adds up an entire row, so it seems you must see the whole row before normalising anything.

The **online softmax** removes that difficulty. It works through a row block by block, keeping two running numbers: the largest value seen so far, m, and a running sum of exponentials, ℓ, measured relative to m. When a new block brings a larger maximum, the old sum is rescaled to match:

$$m^{\text{new}} = \max(m, \tilde{m}), \qquad \ell^{\text{new}} = e^{\,m - m^{\text{new}}}\,\ell \;+ \sum_{\text{new block}} e^{\,z_i - m^{\text{new}}}$$

Here $\tilde{m}$ is the largest value in the new block.

A worked example

Take the scores $z = (1, 3, 2, 5)$, in two blocks of two.

1. **Block 1 is (1, 3).** Its maximum is 3, so $m = 3$ and $\ell = e^{1-3} + e^{3-3} = 1.1353$.
2. **Block 2 is (2, 5).** The maximum rises to 5. Rescale the old sum by e^{3-5} and add the new terms: $\ell = e^{-2} \times 1.1353 + e^{2-5} + e^{5-5} = 1.2034$.
3. **Normalise.** Each probability is $e^{z_i - m}/\ell$. That gives $(0.0152, 0.1125, 0.0414, 0.831)$.

PyTorch's own softmax gives exactly the same four numbers. And $m + \ln \ell = 5.1852$ equals the log-sum-exp of the row, which is the one number FlashAttention saves for the backward pass.

FlashAttention applies the same rescaling to the attention *output*, a running weighted sum of values, as each block of keys arrives. That is the whole algorithm. The block sizes and kernel details are engineering built around this recurrence.

Checking the claims

The exercise reproduces the algorithm and checks each claim:

- **Exactness.** Tiled attention matched standard attention to within 6.7×10^{-16}, for every tile size tried, with and without a causal mask.
- **Memory.** The largest block of scores held at once stayed at 256 numbers, while the full matrix grew from 1,024 numbers at $T = 32$ to 65,536 at $T = 256$.

Sequence length T	Full score matrix	Largest block held at once
32	1,024	256
64	4,096	256
128	16,384	256
256	65,536	256

The analysis, and a claim careful readers get wrong

What makes this a paper rather than a clever trick is the analysis of memory traffic. With SRAM of size M and head dimension d, standard attention moves on the order of $Td + T^2$ numbers. FlashAttention moves on the order of $T^2 d^2 / M$. The paper also proves a lower bound: no exact attention algorithm can do better for all values of M. A speed-up together with a matching lower bound is a much stronger result than a speed-up alone.

Now take the ratio of the two costs and let T grow. The T^2 terms cancel, and the ratio settles at about M/d^2, a constant. The exercise's simplified traffic model shows exactly that:

Sequence length T	Standard traffic ÷ FlashAttention traffic
1,024	4.2
4,096	5.7
16,384	6.0
limit	about 6.1

So FlashAttention's advantage in memory traffic is a large constant factor, not something that keeps growing with sequence length. The paper reports about 9 times less traffic in one measured case; the exercise's simpler model gives about 6. Either way, it is a constant. This is precisely the kind of point question 7 in section 3 is asking for.

The backward pass, and a result from module 01

The backward pass needs the attention probabilities, which were never stored. FlashAttention recomputes them from the queries, the keys and the saved log-sum-exp of each row. Recomputing turns out cheaper than storing, because the stored matrix would have to cross main memory twice.

The backward pass also needs, for each row, the dot product of the probabilities with their incoming gradient. That seems to require the full matrix. It does not, because of an identity you derived in module 01, section 5:

$$\text{rowsum}(\bar{P} \odot P) = \text{rowsum}(\bar{O} \odot O)$$

The bars mean gradients of the loss, as in module 01, and $\odot$ multiplies entry by entry.

Both sides of this equation use only small, per-row quantities. If you did exercise 01, step 6b, you have already proved the lemma this paper depends on.

What to be sceptical about

A careful reader also notices the limits of the evidence:

- The speed-ups are reported for particular sequence lengths and head sizes. They do not extend freely to others. FlashAttention-2 exists partly because the first version used the GPU poorly at some shapes.
- The "2 to 4 times faster" figures compare against a particular PyTorch implementation of the time, not against the best possible one.
- The memory claim is about attention's activation memory, not the total memory of a model. A quick reader can confuse the two.

None of this makes the paper weak. It is what a careful reader should be able to say about it.

> **In the interview:** name the key idea (online softmax), the strongest evidence (exactness plus the lower bound), and one precise limitation (the traffic advantage is a constant factor). That structure works for any paper.

---

6. Presenting your own research

The other half of this round is usually your own work. Rehearse a version that lasts about 90 seconds, with five parts:

1. **The problem**, and why it mattered, in one sentence without jargon.
2. **Why it was hard**: what the obvious approach fails to do.
3. **What you did**: the key idea first, then how it works.
4. **How you know it worked**: the experiment that could have proved you wrong.
5. **What you would do differently.** This question separates researchers from students, and "nothing" is the wrong answer.

Be precise about your own role in joint work. Interviewers detect overclaiming quickly, and it is very damaging. Underclaiming is also a mistake: "I only ran the experiments" undersells real work, because in an empirical field, running the experiments is the work. Say exactly what you did.

Recap

- The round measures judgement, not summary. Say what you think, with evidence.
- Read in three passes. Pass 3, rebuilding the work, is what makes an answer convincing.
- Prepare answers to the ten standard questions, and criticise specifically and in proportion.
- FlashAttention's key idea is the online softmax, which normalises a row block by block, exactly.
- Its memory-traffic advantage is a large constant factor, not one that grows with sequence length.
- Present your own research in five parts, and state your own contribution precisely.

The exercise: pass 3 on FlashAttention

Before you start. Read the paper first: Dao et al. (2022), arXiv:2205.14135. Do passes 1 and 2 before opening the notebook. The notebook is pass 3, and it cannot replace the reading.

What you will build. The exercise has two halves.

- **Part A, in code:** the online softmax, tiled attention, the memory-traffic analysis, and the backward pass that recomputes probabilities instead of storing them. Every claim is checked exactly.
- **Part B, in writing:** a structured critique of the paper, answering ten questions in prose. This half is closest to the interview itself.

Do not expect speed from part A. Python loops over blocks cannot beat a single optimised library call, however good the algorithm. What you are reproducing is exactness and bounded memory, not speed.

Time and setup. Allow three hours for part A and one to two hours for part B, plus the time to read the paper. The setup is the same as for module 00.

How to run it. Open `07-research-paper-analysis/exercise.ipynb`, or run `python exercise.py` from inside the module folder. Write your part B answers directly in the notebook.

How the checks work. They work as in module 00. Before you start, each part A step reports that it is not implemented:

```
[FAIL] A1  -- NotImplementedError:

0/1 passed.  Still failing: A1
```

Step map. This table links each step to the section of the article it practises.

Step	You write	The idea it practises	Section
A1	`online_softmax_stats`, `online_softmax`	the key idea	5
A2	`flash_attention_forward`	tiled, exact attention	5
A3	(measurement code is provided)	memory that does not grow with T	5
A4	`standard_attention_hbm_words`, `flash_attention_hbm_words`, `io_reduction_limit`	the traffic analysis	5
A5	`flash_attention_backward`	recomputation and the module 01 identity	5
B1–B10	written answers	reading and criticising	2 to 4

Watch for one trap in A2. With a causal mask, a whole block of scores can be masked out, which makes its maximum $-\infty$. Guard the rescaling so you never compute $-\infty - (-\infty)$, which is NaN.

When you are done. Your online softmax matches PyTorch's for every block size, including inputs that would overflow a naive softmax. Your tiled attention matches the standard version to within 10^{-13}. Your traffic ratio converges to a constant, and your backward pass matches autograd.

If you get stuck.

- For A1, reproduce the worked example in section 5 by hand first, then in code.
- For A2, start without a causal mask and with a single block, which must equal ordinary attention.
- For A5, check that you can rebuild the probability matrix from the queries, keys and log-sum-exp, before writing the rest of the backward pass.
- If you are still stuck, `solution.py` has a complete reference for part A.

Want more? The notebook ends with four stretch goals. They are a tiled backward pass, a custom autograd function checked with `gradcheck`, a close reading of FlashAttention-2, and the same exercise on a different paper.

Going deeper

Start here

- Keshav (2007), *How to Read a Paper*, ACM SIGCOMM Computer Communication Review 37(3). Two pages, and the source of the three-pass method in section 2. Read it before anything else.

The paper for this module

- Dao, Fu, Ermon, Rudra & Ré (2022), *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*, NeurIPS. arXiv:2205.14135. Required reading before the exercise.

- Milakov & Gimelshein (2018), *Online normalizer calculation for softmax*. arXiv:1805.02867. Four pages. The recurrence FlashAttention is built on.
- Rabe & Staats (2021), *Self-attention Does Not Need O(n^2) Memory*. arXiv:2112.05682. An earlier, independent version of the memory argument. Reading it answers a strong interview question: what exactly did FlashAttention add?

What came next

- Dao (2023), *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*. arXiv:2307.08691. Read it second, and ask what it reveals about the first version's weaknesses.
- Shah et al. (2024), *FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision*. arXiv:2407.08608.

The approximate methods it displaced

- Wang et al. (2020), *Linformer: Self-Attention with Linear Complexity*. arXiv:2006.04768.
- Choromanski et al. (2021), *Rethinking Attention with Performers*, ICLR. arXiv:2009.14794.
- Tay et al. (2022), *Efficient Transformers: A Survey*, ACM Computing Surveys 55(6). arXiv:2009.06732. A map of the approximate-attention methods that FlashAttention largely made unnecessary.

On research more broadly

- Hamming (1986), *You and Your Research* — https://www.cs.virginia.edu/~robins/YouAndYourResearch.html. A classic talk on choosing problems worth working on.
- Whitesides (2004), *Whitesides' Group: Writing a Paper*, Advanced Materials 16(15). Reading papers becomes easier once you know how they are built.
- DAIR.AI, *ML Papers Explained* — https://github.com/dair-ai/ml-papers-explained. Short explanations of many machine-learning papers, useful for pass 1.

Next: module 08 moves to questions without numerical answers: AI safety and alignment.